

NK-7 — Guía de Presentación del Equipo

Ingeniería de Software — Exposición del Videojuego

Duración total: 10–15 minutos | 4 integrantes

CONTEXTO GENERAL (todos deben saber esto)

NK-7 es un plataformero 2D de acción y puzzle en **Godot 4.6** con GDScript.

53 scripts · 37 escenas · 167 assets · 6 singletons · 5 niveles

LA HISTORIA (solo lo que se ve en el juego actual)

KAI es un técnico de mantenimiento atrapado en una instalación industrial tras un fallo del sistema de control. Las salidas están bloqueadas, los láseres en modo peligroso y los robots de seguridad patrullan los pasillos. El jugador debe avanzar por los sectores usando el conocimiento técnico de Kai: hackear sistemas, reparar puertas y esquivar o eliminar robots.

Lo que se ve en el demo:

- **Nivel 1 (Sector B):** Tres pisos de la instalación, láseres, robots Ukibuki, un switch que desactiva un láser, y una puerta de salida al siguiente sector
- **Nivel 2 (Sector 7-F):** Continuación del recorrido con más enemigos

La historia completa está documentada en el repositorio, pero en la exposición basta con describir lo que el docente va a ver en pantalla.

DISTRIBUCIÓN — 3 min por persona aprox.



1. TATIANA — Historia + Arquitectura

~3 min

Historia (1.5 min)

"Buenos días, vamos a presentar NK-7, un videojuego 2D de acción y puzzle desarrollado en Godot 4.6."

El juego:

"NK-7 es un plataformero 2D donde controlamos a **Kai**, un técnico de mantenimiento atrapado en una instalación industrial tras un fallo del sistema de control. Las salidas están bloqueadas, los robots de seguridad patrullan los pasillos y los láseres están en modo peligroso. El jugador avanza por los sectores usando el conocimiento técnico de Kai: hackea sistemas, repara puertas y enfrenta a los robots Ukibuki."

Lo que vamos a mostrar hoy:

"En el demo tienen el Sector B — tres pisos de la instalación con láseres, robots, un puzzle de interruptor y una puerta que lleva al siguiente sector. La historia completa está documentada en el repositorio."

Por qué es interesante como proyecto de software:

"Cada mecánica tiene una justificación narrativa. Las puertas dañadas existen porque la instalación colapsó. Los láseres están en modo peligroso porque el protocolo de seguridad los reconfiguró. La stamina representa el esfuerzo físico de un técnico escapando. Eso hace que el diseño del juego y el diseño del software estén alineados."

Arquitectura (1.5 min)

Stack tecnológico:

Tecnología	Versión	Por qué la usamos
Godot Engine	4.6 stable	Open source, sistema de escenas nativo para 2D, señales que implementan Observer de forma nativa
GDScript	2.0	Lenguaje propio de Godot, tipado opcional, sintaxis similar a Python, ideal para prototipar rápido
Jolt Physics	Integrado	Motor de física de alto rendimiento, más preciso que el motor por defecto de Godot para plataformeros
Forward+ Renderer	Godot 4.6	Renderizado moderno con soporte de luces dinámicas (usadas en los robots y puertas)
Git + GitHub	—	Control de versiones, ramas por integrante, commits semánticos

"Elegimos Godot sobre Unity porque es completamente open source, no tiene restricciones de licencia, y su sistema de escenas hace que cada objeto del juego sea independiente y reutilizable. GDScript nos permitió iterar rápido sin la complejidad de C#."

El proyecto tiene **3 capas**:

```
PRESENTACIÓN → Escenas .tscn + UI
LÓGICA       → Scripts de personajes, enemigos, objetos
DATOS/ESTADO → 6 Autoloads Singleton (accesibles desde cualquier script)
```

Singleton	Qué hace
GameManager	Guardado, transiciones, score, muertes
AudioManager	Música y efectos de sonido
LevelStateManager	Persiste estado de niveles
AchievementManager	Logros y contadores

Singleton	Qué hace
<code>GameState</code>	Estado global de sesión
<code>NK7Theme</code>	Tema visual global

"Estos singletons reducen el acoplamiento — cualquier script accede a GameManager sin referencias directas entre escenas. Con 37 escenas en el proyecto, esto es crítico."

2. STEVEN — Mecánicas del Jugador + Guardado

~3 min

Sistema de recursos (1.5 min)

GDScript y tipado estático — el proyecto usa tipado explícito en todas las variables críticas:

```
var health : float = MAX_HEALTH # float para daño fraccionario
var stamina : float = MAX_STAMINA # degradación suave por delta
var energy : float = MAX_ENERGY # consumo por frame al hackear
```

Esto mejora el rendimiento y detecta errores en tiempo de edición, no en ejecución.

Kai tiene **3 recursos** que el jugador gestiona al mismo tiempo:

Recurso	Representación	Se gasta con...	Se recupera con...
♥ Salud	3 corazones	Daño de enemigos/láseres	No se regenera
⚡ Stamina	Barra verde/azul	Correr, escalar, empujar	Estar quieto o agachado
📄 Energía	(interna)	Hackear, atacar	Esperar o agacharse

"Esto crea decisiones constantes: ¿corro y llego más rápido pero sin stamina? ¿hackeo ahora o espero a tener más energía?"

Stamina en detalle:

- Correr: -18 pts/s · Escalar rápido: -10 pts/s · Empujar: -8 pts/s
- Regen parado: +12 pts/s (delay 1.2s) · Regen agachado: +28 pts/s (inmediata)
- Si se agota: no puede correr hasta recuperar 20 pts

Sistema de corazones:

- 3 corazones = 100 HP · I-frames de 1.5s tras cada golpe
- Al recibir daño: animación de bounce + textura de daño por 0.4s
- HP = 0 → Game Over con fade-in, viñeta roja y shake de pantalla

Controles (30 seg)

Acción	Tecla	Nota
Mover	A / D	
Correr	Shift	Consume stamina
Saltar	Espacio	Más fuerza si corre
Agacharse	Ctrl	Regenera recursos
Escalar	W / S	Detecta escaleras automáticamente
Interactuar	E	Puertas, switches
Hackear	Q	Consume energía
Atacar	F	Rango 80px, daño 30

Guardado y checkpoints (1 min)

3 slots de guardado — cada uno guarda: nivel actual, checkpoint, muertes, tiempo, score y estado del nivel (qué enemigos cayeron, qué puertas se abrieron).

SmartCheckpointSystem — checkpoints inteligentes:

- No son posiciones fijas
- Al morir, busca la **zona segura más cercana** al punto de muerte
- Hace un raycast hacia abajo para asegurarse de que hay suelo sólido
- Evita spawns en el aire o dentro de paredes



3. SEBASTIAN — IA de Enemigos + Sistema de Puertas

~3 min

IA de los Ukibuki (1.5 min)

Herramientas de Godot usadas:

- **PhysicsRayQueryParameters2D** — para el Line-of-Sight del robot
- **CharacterBody2D** — cuerpo físico del robot con `move_and_slide()`
- **PointLight2D** — luz de estado que cambia de color según el estado de la FSM
- **Señales de Godot** — `player_detected`, `attacked`, `destroyed` para comunicar eventos sin acoplamiento

Los **Ukibuki** son robots de seguridad con una **máquina de estados finita (FSM)**:

```

IDLE → PATROL → ALERT → CHASE → ATTACK
                        ↓
                    DAMAGED → DESTROYED
  
```

Estado	Qué hace
IDLE	Quieto, detecta jugador en rango 320px
PATROL	Patrulla ±200px, pausa 0.35s al girar
ALERT	Detectó jugador, se orienta
CHASE	Persigue a 1.4× velocidad, recuerda última posición 2.5s
ATTACK	Verifica LOS, dispara, cooldown 2.2s
DAMAGED	Pausa 0.5s tras recibir daño
DESTROYED	Explosión, desaparece tras 1.2s

Line-of-Sight: Usa un **Raycast** hasta el jugador. Si hay una pared entre ellos, NO dispara — el jugador puede usar la geometría como cobertura.

Al destruir un Ukibuki: +500 puntos, camera shake, registra derrota en **LevelStateManager** (persiste entre sesiones).

Sistema de puertas (1.5 min)

Herramientas de Godot usadas:

- **AnimatedSprite2D** — animaciones de apertura/cierre con frames del spritesheet **doors.png**
- **Area2D** — zona de detección del jugador (sin colisión física, solo trigger)
- **StaticBody2D** — colisión física de la puerta cerrada, se desactiva al abrir
- **AtlasTexture** — recorte preciso de frames del spritesheet (265×300px por frame, 9 frames totales)
- **Tween** — animaciones de brillo y pulso al acercarse a la puerta

6 tipos de puertas, cada una consume un recurso distinto:

Tipo	Cómo se abre	Recurso
Normal	Automática	Ninguno
Dañada	[F] Reparar	Stamina
Bloqueada	[E] Hackear	Energía
Electrificada	[F] Reparar → [E] Hackear	Stamina + Energía
De nivel	[E] Entrar	Ninguno (transición)
Con keycard	Coleccionable	Keycard

La **puerta electrificada** tiene 7 estados internos: **LOCKED_DAMAGED** → **REPAIRING** → **LOCKED_FIXED** → **UNLOCKING** → **OPENING** → **OPEN** → **ELECTRIFIED**

"Cada puerta consume un recurso diferente de Kai. Esto conecta el diseño de niveles con la gestión de recursos — si llegás sin energía, no podés hackear la puerta."

4. HECTOR (TÚ) — Patrones de Diseño + Demo + Cierre

~3 min (lo más liviano)

Patrones de diseño (1 min)

"Antes de ver el juego en acción, les muestro los patrones de diseño que aplicamos."

Patrón	Dónde
Singleton	Los 6 autoloads — un punto de acceso global sin dependencias directas
FSM	IA del Ukibuki (7 estados) y puertas electrificadas (7 estados)
Observer	Señales de Godot: <code>door_opened</code> , <code>player_detected</code> , <code>level_completed</code>
Template Method	<code>level_base.gd</code> define el esqueleto; cada nivel implementa su lógica específica
Strategy	Comportamiento de movimiento cambia según estado: correr, agacharse, escalar

"El más importante es el Singleton — con 37 escenas que comparten estado, sin él tendríamos dependencias circulares por todos lados."

Herramientas visuales y de UI:

Herramienta	Uso en el juego
GLSL Shaders	Efecto de distorsión de calor en el menú principal, scanlines, borde neón en botones
CanvasLayer	HUD siempre encima del juego (layer 5), transiciones de nivel (layer 128)
Tween	Todas las animaciones de UI: fade in/out, bounce de corazones, pulso de puertas
ParallaxBackground	Fondo con profundidad en los niveles
ColorRect + CanvasLayer	Fade negro entre niveles, viñeta roja en Game Over

"Los shaders están escritos en GLSL y se integran directamente en Godot como recursos `.gdshader`. El efecto de calor del menú usa distorsión de onda sinusoidal sobre el fondo."

Demo en vivo (1.5 min)

Muestra esto en orden:

1. **Splash screen** → cualquier tecla → menú principal
2. **Iniciar Partida** → escribe nombre → nivel 1
3. **Corre con Shift** → señala que la barra de stamina baja
4. **Toca un láser** → pierde un corazón con animación
5. **Acércate a un Ukibuki** → PATROL → ALERT → CHASE → ATTACK
6. **[E] en el switch** → láser del piso 3 se apaga
7. **Puerta de salida** → [E] → animación → transición al nivel 2

Cierre (30 seg)

"NK-7 aplica ingeniería de software de forma práctica: arquitectura en capas, patrones reconocidos y sistemas desacoplados. Si mañana queremos agregar un nuevo enemigo o tipo de puerta, no tocamos código existente — solo lo extendemos. El repositorio tiene commits por integrante, documentación técnica en [docs/](#) y un README completo. Estamos abiertos a preguntas."

PREGUNTAS FRECUENTES

¿Por qué Godot y no Unity?

Open source, sistema de escenas más limpio para 2D, y las señales implementan Observer de forma nativa.

¿Patrón más importante?

Singleton — con 37 escenas compartiendo estado, es lo que evita dependencias circulares.

¿Cómo funciona el guardado?

3 slots `.dat`. Guarda nivel, checkpoint, muertes, tiempo, score y estado del nivel. `LevelStateManager` restaura qué enemigos cayeron y qué puertas se abrieron.

¿Cómo funciona el checkpoint?

`SmartCheckpointSystem` busca la zona segura más cercana al punto de muerte y hace raycast hacia abajo para encontrar suelo sólido.

¿Cómo está implementada la IA?

FSM con 7 estados. Transiciones basadas en distancia y raycast de línea de visión. Recuerda la última posición del jugador 2.5s antes de volver a patrullar.

¿Es escalable el sistema de puertas?

Sí. Cada puerta es una escena independiente. Para agregar un tipo nuevo, se crea una escena nueva que usa los métodos públicos de Kai (`consume_energy()`, `use_tool()`). Sin tocar código existente.

RESUMEN RÁPIDO

Motor	Godot 4.6, Forward+ Renderer
Lenguaje	GDScript 2.0 (tipado estático)
Física	Jolt Physics (integrado en Godot 4.6)
Shaders	GLSL — distorsión de calor, scanlines, borde neón
Control de versiones	Git + GitHub, ramas por integrante
Género	Plataformero 2D acción/puzzle

Protagonista	Kai, técnico de mantenimiento
Niveles	5 + epílogo (2 jugables en el demo)
Recursos del jugador	Salud (3 corazones), Stamina, Energía
Enemigos	Ukibuki — FSM 7 estados + Line-of-Sight
Patrones	Singleton, FSM, Observer, Template Method, Strategy
Guardado	3 slots, persistencia de estado de nivel
Scripts	53 <code>.gd</code> · Escenas: 37 <code>.tscn</code> · Assets: 167 <code>.png</code>

NK-7 — Ingeniería de Software